PORTFOLIO.

AI Backend & Data Developer 서다은 포트폴리오

 SEO DAEUN

 010-9740-4640

 sde4502@naver.com

 https://github.com/Seodaeun-hub

ABOUT ME.

불편을 그냥 지나치지 않고,
문제는 끝까지 확인합니다.

주변 사람의 작은 고민에서 아이디어를 찾고, 예상과 다른 결과가 나오면 “왜?”를 따라 원인을 좁혀갑니다.
맡은 기능은 구현에서 끝내지 않고 실제로 동작하는 순간까지 확인해야 마음이 놓입니다. 혼자 정답을 정하기보다 팀과 기준을 맞추며 더 나은 결과를 만들어갑니다.
관찰하는 사람
끝까지 확인하는 사람
함께 기준을 맞추는 사람

OBSERVE | 작은 불편을 발견합니다.

PERSIST | 원인을 끝까지 확인합니다.

TOGETHER | 팀과 기준을 맞춥니다.

CORE COMPETENCIES.

Backend Architecture

FastAPI 기반 API와 비동기 작업 흐름을 설계하고 실제 업무 처리까지 연결

Async & Realtime

Celery, Redis, SSE, Pub/Sub을 활용한 비동기 처리 및 실시간 상태 전달

Reliability

Retry, FAILED 상태, DLQ, HITL 등 정상 흐름뿐 아니라 실패 이후 처리까지 설계

AI Service Integration

AI Model/Agent의 결과를 DB, API, 실제 서비스 로직과 연결



SEO DAEUN

2001.01.03

EDUCATION

2023. 08 인덕대학교 정보통신공학과 중퇴

2024. 03 건국대학교 컴퓨터공학과 학사 편입

2026. 08 건국대학교 컴퓨터공학과 졸업

EXPERIENCE

2026. 03 ~ 2026. 09 KT AIVLE SCHOOL 9기

LICENSE / AWARD

2023. 12. 15 컴퓨터활용능력 1급

2026. 04. 25 AICE Associate

2026. 09. 06 TOEIC Speaking AL

2026. 09. 03 AIVLE Big Project Excellence

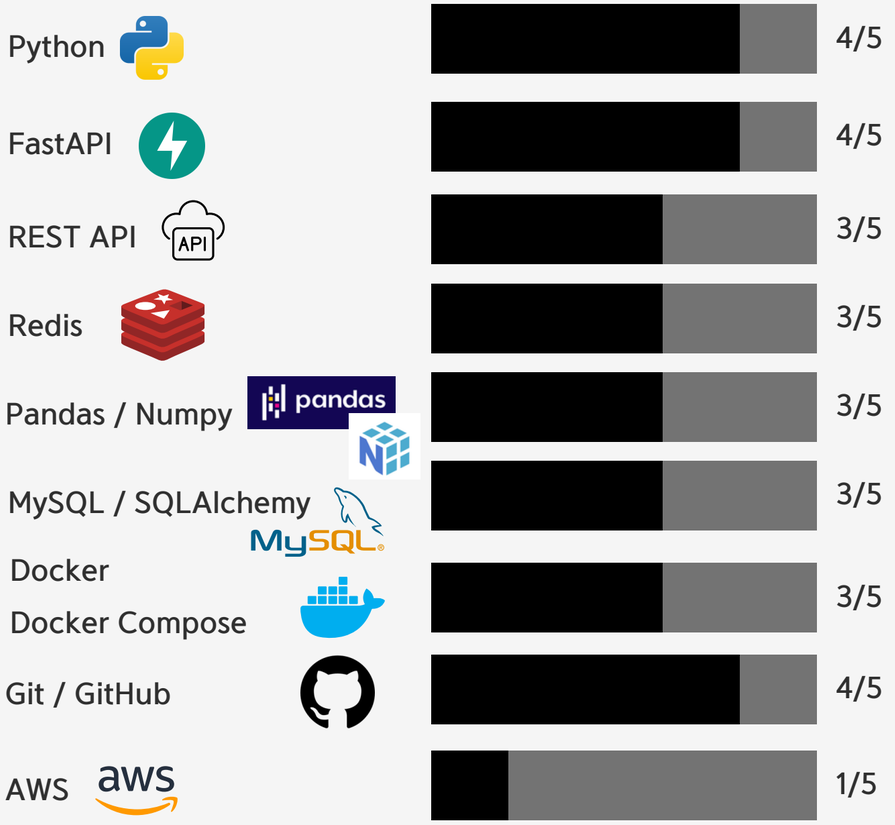
SKILLS

BACKEND ARCHITECTURE & OPERATION

- Async Processing: Celery, Redis
- Realtime: SSE, Redis Pub/Sub
- Reliability: Retry, FAILED state, DLQ, HITL
- Monitoring: Flower
- Cloud: AWS EC2, S3

SKILLS

BACKEND / API



AI INTEGRATION

- LangGraph
- AI Agent
- PyTorch
- EfficientNet-B0

MAIN PROJECT.1

NEWZED

개발 기간 : 2026.07 ~ 2026.09

개발 인원 : 6명

담당 역할 : Backend

주요 담당 : 검수 API, Celery·Redis 오케스트레이션, SSE, HITL·RBAC

개발 환경 : Python, FastAPI, PostgreSQL, Redis, Celery, Docker

연동 기술 : LangGraph, REST API, Redis Pub/Sub, SSE

https://github.com/jmkyong9393/wms-core-backend

1.프로젝트 소개

Multi-Agent B2B AI WMS

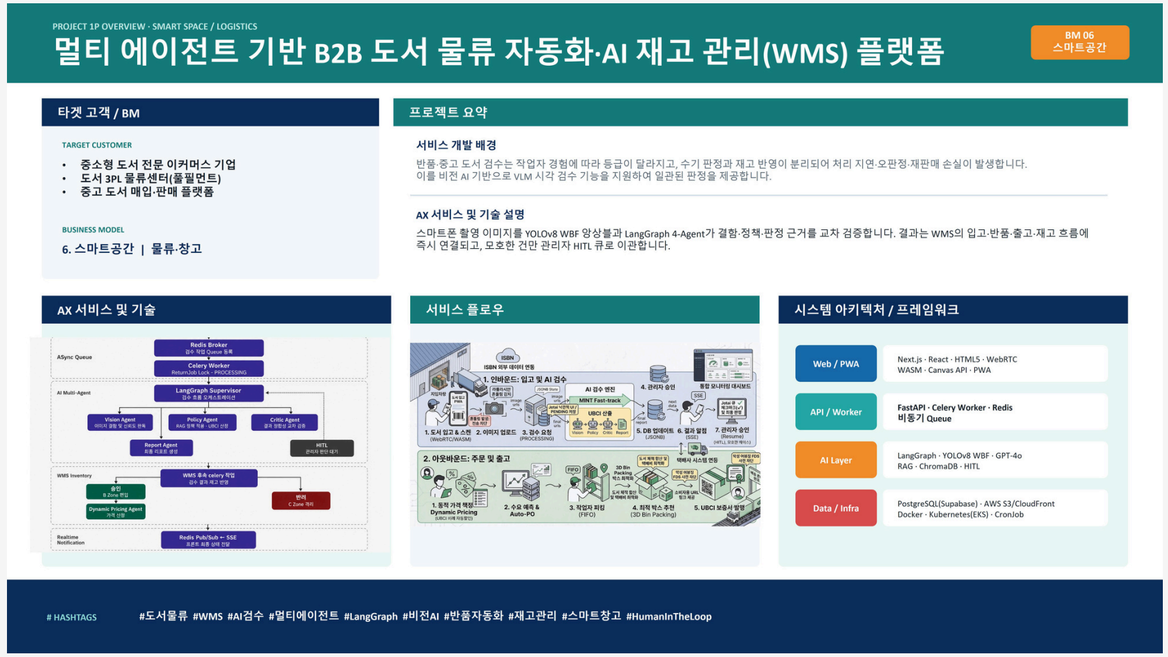
반품 도서 이미지를 AI로 검수하고, 판정 결과를 WMS 입고·반려 처리까지 연결하는 B2B 물류 자동화 플랫폼입니다.

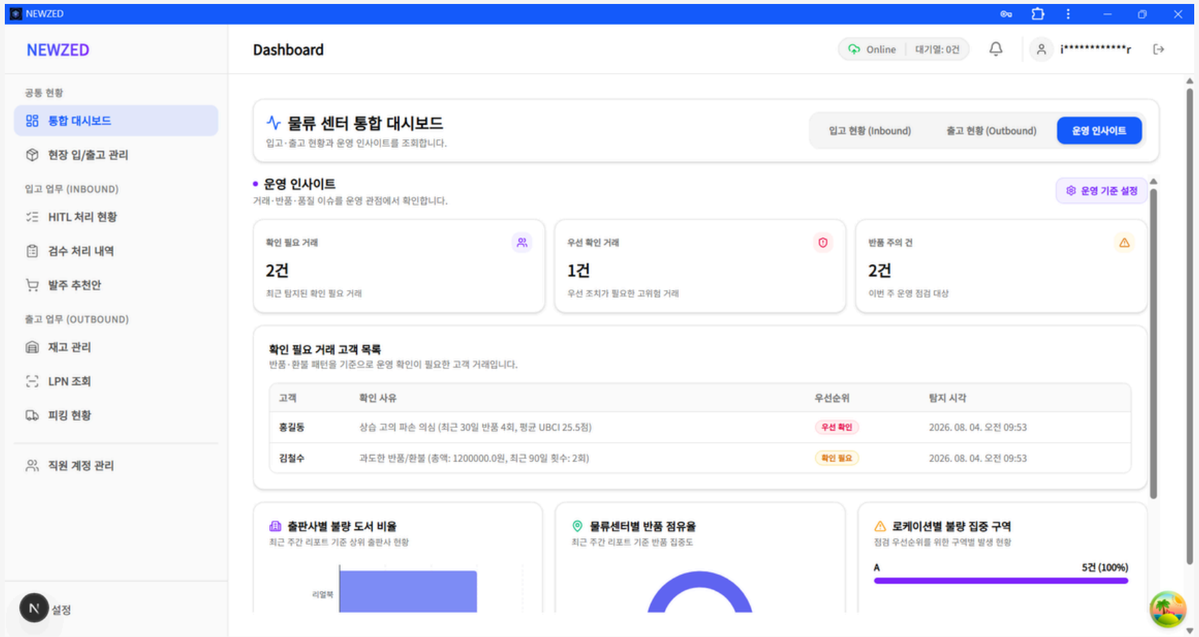
프로젝트 설명

작업자가 모바일에서 반품 도서 이미지를 등록하면 비동기 AI 검수가 도서의 상태와 UBCI 점수를 분석합니다. 검수 결과에 따라 자동 승인·반려하거나 관리자의 HITL 판정을 거쳐 WMS 후속 처리를 실행하며, 전체 진행 상태를 관리자 화면에 실시간으로 전달하도록 구현했습니다.

역할 한 줄 요약

AI 검수 요청부터 WMS 후속 처리와 실시간 상태 전달까지 이어지는 백엔드 오케스트레이션을 담당했습니다.





2.기여한 부분

검수 요청부터 WMS 처리까지 백엔드 흐름 구현

저는 **FastAPI 기반 Core API와 Celery·Redis 비동기 처리 구조를 담당**했습니다.

AI 판정 결과를 저장하는 데서 끝나지 않고, 관리자 검토와 WMS 입고·반려 처리, 실시간 상태 전달까지 하나의 서비스 흐름으로 연결했습니다.

기여 1. 검수 API 및 상태 관리

- POST /api/v1/inspections 검수 생성 API 구현
- 검수 요청과 Celery Task 연결
- 검수 진행 상태 및 결과 조회 API 구현
- PENDING → PROCESSING → APPROVED/REJECTED 상태 흐름 관리
- 예외 상황에 FAILED, 관리자 확인이 필요하면 HITL_REQUIRED로 분리

기여 2. 비동기 오케스트레이션

- Redis Broker와 Celery 기반 AI 검수 작업 구성
- AI 검수 결과 저장 후 WMS 후속 작업 실행
- WMS 일시 장애에 대한 재시도 정책 적용
- AI Task와 WMS Task 분리
- 실패 단계와 재시도 횟수를 구분해 기록
- 중복 처리를 방지하기 위한 작업 식별자 관리

기여 3. 실시간 상태 전달 및 관리자 처리

- Redis Pub/Sub 기반 SSE 상태 전달 구현
- 검수 상태와 진행률을 실시간으로 프론트에 전달
- 관리자 승인·반려·재검수 HITL API 구현
- MASTER, ADMIN, WORKER 역할 기반 접근 제어
- 사용자 상태가 ACTIVE인 경우에만 기능을 사용할 수 있도록 검증

3.트러블 슈팅 1/2

WMS 장애 시 AI 검수가 반복 실행되던 문제

HTTP 503 오류 시험에서 AI 실행 4회 → 1회
완료된 AI 검수 재실행 3회 → 0회
AI 결과를 보존하고 실패한 WMS 작업만 재시도

상황

기존에는 LangGraph 기반 AI 검수와 WMS API 호출을 **하나의 Celery Task에서 처리**했습니다. AI 검수가 완료됐더라도 WMS API에서 일시적인 오류가 발생하면 self.retry()가 전체 작업을 다시 실행했습니다. 이로 인해 **복구가 필요한 부분은 WMS 호출뿐인데도 이미 완료된 AI 검수가 반복되는 문제**가 있었습니다.

원인

기존 구조
AI 검수 → WMS 호출 → HTTP 503 발생 → Celery Task 전체 재시도 → AI 검수부터 다시 실행

재시도 범위가 WMS 호출이 아니라 전체 검수 작업으로 설정된 것이 원인이었습니다.

해결

AI 검수와 WMS 처리를 별도의 Celery Task로 분리했습니다. AI 결과를 DB에 먼저 저장한 뒤 process_wms_action을 등록하고, WMS에서 일시적인 오류가 발생하면 저장된 AI 결과를 이용해 WMS Task만 재시도하도록 변경했습니다.

개선 구조
process_inspection → AI 검수 → AI 결과 DB 저장 → process_wms_action 등록 → WMS 호출만 재시도

검증 결과

강제 HTTP 503 환경에서 재시도 간격 5초, 최대 재시도 3회로 비교했습니다.

측정 항목	개선 전	개선 후
AI 실행	4회	1회
불필요한 AI 재실행	3회	0회
WMS 실행	4회	4회
WMS 재시도	3회	3회

배운 점

재시도 로직에서는 횟수뿐 아니라 **재시도의 범위를 설계하는 것이 중요**하다는 점을 배웠습니다. 외부 시스템 장애가 이전 처리 단계까지 다시 실행시키지 않도록 작업을 분리하고, 완료된 결과를 먼저 저장해야 안정적으로 복구할 수 있다는 것을 확인했습니다.

3.트러블 슈팅 2/2

반복 DB 조회 98.2% 감소
상태 변경이 있을 때만 이벤트 전달
연결 직후 현재 상태를 조회해 결과 확인 보완

SSE 상태 전달 과정에서 반복되던 DB 조회 개선

상황

초기 SSE 구현에서는 검수 상태를 실시간으로 전달하기 위해 1초마다 ReturnJob을 조회했습니다. 연결된 사용자가 많아질수록 각 SSE 연결에서 동일한 DB 조회가 반복되어 불필요한 부하가 증가할 수 있는 구조였습니다.

원인

기존 Polling 방식
SSE 연결 → ReturnJob 조회 → 1초 대기 → ReturnJob 다시 조회 → 상태가 종료될 때까지 반복

상태 변경 여부와 관계없이 **연결이 유지되는 동안 DB를 계속 조회하는 방식이 원인**이었습니다.

해결

Worker가 상태를 변경할 때 **Redis 채널에 이벤트를 발행하고, SSE 서버는 해당 채널을 구독해 이벤트가 발생했을 때만 프론트로 전달하도록 변경**했습니다.

개선된 Pub/Sub 방식
Worker 상태 변경 → Redis 채널에 이벤트 발행 → SSE 서버가 이벤트 수신 → 프론트에 상태 전달

다만 Redis 이벤트가 발행되기 전에 사용자가 연결하거나, 이미 작업이 완료된 뒤 연결할 수도 있기 때문에 **연결 직후 DB 상태 확인 1회는 유지**했습니다.

검증 결과

동일한 ReturnJob을 PROCESSING 상태로 유지하고, Polling과 Redis Pub/Sub SSE를 각각 60초 연결한 뒤 API 로그의 DB 조회 횟수를 비교했습니다.

측정 항목	개선 전	개선 후
SSE 연결 시간	60초	60초
ReturnJob 조회	57회	1회
감소한 조회	-	56회
DB 조회 감소율	-	**98.20%**

배운 점

실시간 기능은 짧은 주기로 데이터를 반복 조회하는 것만으로도 구현할 수 있지만, 연결 수가 늘어나면 DB 부하도 함께 증가합니다. 상태 변경 주체가 이벤트를 발행하고 구독자가 이를 전달받도록 구조를 바꾸면서, **실시간성과 데이터베이스 효율을 함께 고려하는 방법을 배웠습니다.**

MAIN PROJECT.2

CRYPTOVA

개발 기간 : 2025.09~2026.05

개발 인원 : 2명

담당 역할 : Full-stack 공동 개발

주요 담당 : 자동매매 실행 로직, 포지션 상태 관리, History·Backtest, AI 설명 연동

Backend : Python, FastAPI, SQLAlchemy, MySQL, APScheduler

Frontend : React, JavaScript, Lightweight Charts

External API : Bybit Public API, Gemini API, AI Inference Server



https://github.com/Seodaeun-hub/Cryptova.git



https://www.youtube.com/watch?si=wnuErqak-WE5TG42&v=uUBxkVrYecI&feature=youtu.be

1. 프로젝트 소개

AI 기반 암호화폐 자동매매 서비스 Cryptova

AI가 생성한 매매 신호를 사용자 전략과 리스크 기준으로 검증하고, 포지션 진입부터 청산·손익·실행 이력까지 관리하는 자동매매 서비스입니다.

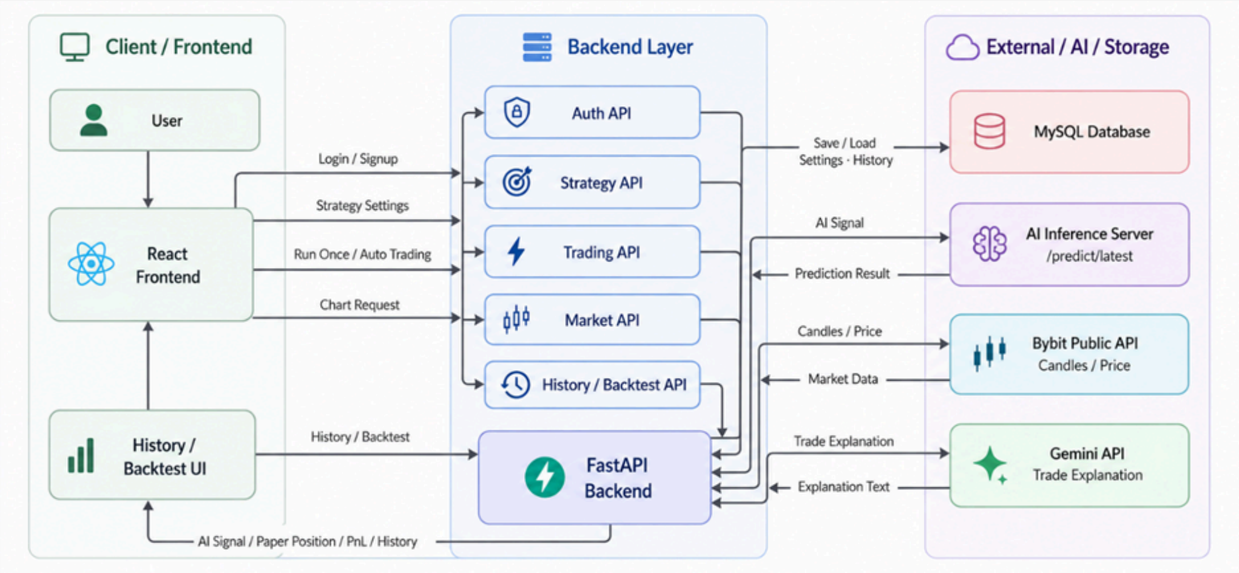
프로젝트 설명

암호화폐 시장 데이터를 기반으로 AI가 LONG, SHORT, HOLD 신호를 생성하면, 사용자 전략과 리스크 조건을 적용해 실제 실행 여부를 판단합니다. 실행된 포지션은 진입·보유·청산 상태로 관리하며, 사용자는 AI 판단 근거와 거래 이력, 백테스트 결과를 확인할 수 있습니다.

역할 한 줄 요약

AI 신호가 사용자 전략 검증을 거쳐 포지션과 실행 이력으로 이어지는 자동매매 서비스 흐름을 구현했습니다.





2.기여한 부분

AI 신호를 실행 가능한 자동매매 흐름으로 연결

단순히 AI가 예측한 매매 방향을 화면에 표시하는 것에서 끝내지 않고, **사용자 설정과 리스크 조건을 반영해 포지션 진입·보유·청산과 실행 이력으로 이어지는 전체 흐름을 구현**했습니다.

시장 데이터 → AI 신호 생성 → 사용자 전략·리스크 검증 → 중복 포지션 확인 → 주문 실행 → 포지션 관리 → PnL·History 저장

기여 1. 자동매매 실행 흐름

- 암호화폐 시장 데이터와 AI 예측 결과 연결
- LONG, SHORT, HOLD 신호 처리
- 사용자 전략과 리스크 조건을 적용한 실행 여부 판단
- Run Once 기반 수동 실행 흐름 구현
- APScheduler 기반 Auto Trading 구현
- API Key 보유 여부에 따른 실행 경로 분리

기여 2. 포지션 상태 관리

- 포지션 진입 시 OPEN 상태와 hold_until 저장
- 보유기간 이전에는 신규 포지션 진입 차단
- 보유기간 이후 기존 포지션 청산
- 청산 가격을 기준으로 PnL 계산
- 포지션 결과를 사용자 자산과 History에 반영

기여 3. 결과 검증과 사용자 설명

- 자동매매 실행 이력 조회 화면 구현
- 백테스트 실행과 결과 시각화
- Gemini 기반 AI Trade Explanation 연동
- AI 신호·confidence·시장 정보·리스크 조건을 설명에 반영
- 외부 AI 설명 API 실패 시 fallback 메시지 처리

3.트러블 슈팅 1/3

- API Key 없이 Paper 주문 흐름 검증
- 실제 거래소 주문 0건
- 신호 생성부터 OPEN 포지션 저장까지 확인

거래소 인증정보 없이 주문 이후 흐름을 검증하기 어려웠던 문제

상황

기존에는 AI 신호가 생성돼도 Bybit API Key가 없는 개발 환경에서는 실제 주문을 실행할 수 없었습니다. 주문 단계에서 예외가 발생하면 이후 포지션 생성과 History 확인까지 이어지는 전체 흐름을 검증하기 어려웠습니다.

원인

기존 구조
AI 신호 → API Key 조회 → 실제 주문 요청 → 포지션 생성

외부 거래소 인증 여부와 서비스 내부의 주문·포지션 관리 로직이 하나의 실행 경로에 결합된 것이 원인이었습니다.

해결

API Key 유무에 따라 실제 주문과 Paper Execution 경로를 분리했습니다. API Key가 없는 경우 공개 시세를 조회해 주문 수량을 계산하고, PAPER_SUBMITTED 상태의 주문과 OPEN 포지션을 저장하도록 구현했습니다.

개선 구조
AI 신호 → API Key 확인 → 공개 시세 조회 → Paper 주문 저장 → OPEN 포지션 생성

실제 주문과 Paper 주문이 동일한 Order·TradingPosition 구조를 사용하도록 구성해 포지션 관리와 History 로직을 공통으로 검증했습니다.

검증 결과

Bybit API Key가 없는 신규 계정에서 고정 LONG 신호를 사용해 검증했습니다.

측정 항목	검증 결과
신호 생성	1건
Paper 주문	1건
주문 상태	PAPER_SUBMITTED
OPEN 포지션	1건
실제 거래소 주문	0건
신호→포지션 처리	187.92ms

배운 점

외부 시스템과 연동하는 서비스에서는 실제 실행 여부와 내부 상태 관리를 분리하는 것이 중요하다는 점을 배웠습니다. 동일한 데이터 구조를 유지하면서 실행 경로만 분리하면, **외부 인증정보가 없는 환경에서도 주문 이후의 핵심 로직을 안전하게 검증할 수 있었습니다.**

3.트러블 슈팅 2/3

반복 실행 시 중복 주문과 포지션이 생성될 수 있는 문제

반복 실행 10/10 차단
추가 주문·OPEN 포지션 0건
평균 17.24ms · p95 32.39ms

상황

자동매매 Scheduler와 수동 run-once 요청이 반복되면 기존 포지션의 보유 기간이 끝나기 전에도 **새로운 AI 신호와 주문이 실행**될 수 있었습니다. 이 경우 같은 사용자와 종목에 **여러 OPEN 포지션이 생성**될 가능성이 있었습니다.

원인

기존 구조
run-once → AI 신호 생성 → 주문 생성 → OPEN 포지션 생성

실행 초기에 기존 OPEN 포지션과 보유 종료 시각을 확인하는 단계가 없었던 것이 원인이었습니다.

해결

AI 서버를 호출하기 전에 사용자와 종목을 기준으로 **OPEN 포지션을 먼저 조회**하도록 실행 순서를 변경했습니다. 현재 시각이 hold_until 이전이면 AI 신호와 신규 주문을 실행하지 않고 SKIPPED_HOLDING으로 종료했습니다.

개선 구조
run-once → OPEN 포지션 조회 → hold_until 확인
→ 보유 기간 유지: SKIPPED_HOLDING
→ 보유 기간 종료: 기존 포지션 청산

차단된 실행도 TradingRun에 기록해 반복 요청의 처리 결과를 확인할 수 있도록 했습니다.

검증 결과

OPEN 포지션이 존재하는 상태에서 run-once를 연속 10회 호출했습니다.

측정 항목	검증 결과
반복 호출	10회
SKIPPED_HOLDING	10회
추가 주문	0건
추가 OPEN 포지션	0건
TradingRun 기록	10건
평균 응답시간	17.24ms
p95 응답시간	32.39ms

배운 점

반복 실행되는 작업에서는 실행 횟수보다 **현재 상태를 먼저 확인**하는 것이 중요하다는 점을 배웠습니다. AI 호출 전에 포지션 상태와 보유 기간을 확인함으로써 불필요한 외부 호출과 중복 주문을 막고, 차단 결과까지 기록해 실행 과정을 추적할 수 있었습니다.

3.트러블 슈팅 3/3

입력 확장 실험을 통해 예측과 위험 제어를 분리하다

입력 확장 → 구간별 성과 불일치
예측과 위험 제어 역할 분리
구간 2 손실률 10.20%p 감소 — 비교 백테스트 기준

상황

차트와 뉴스 데이터를 시간 기준으로 병합한 뒤, 모델에 사용할 입력 컬럼을 선정했습니다. 거래량·변동성·파생상품 정보를 추가하며 팀원과 반복 실험했지만, **같은 입력 조합도 평가 구간에 따라 결과가 달라** 채택 기준을 정하기 어려웠습니다.

원인

[구간별로 일관되지 않은 개선 효과]

추가 정보를 예측 입력에 포함하면 한 구간의 성과는 개선됐지만 다른 구간에서는 악화됐습니다. 정보를 많이 넣는 것만으로는 안정적인 개선을 확보하기 어려웠고, 해당 정보가 **방향 예측에 필요한지, 위험한 거래를 제한하는 데 적합한지 구분**할 필요가 있었습니다.

해결

[예측 입력과 거래 제한 조건 분리]

차트·뉴스 단독 모델과 결합 모델, 입력 컬럼 조합별 결과를 구간별로 비교했습니다. 이후 **기존 예측 입력은 유지하고, 추가 정보는 예측 이후 적용하는 외부 필터로 분리**했습니다. 매수 신호가 발생해도 펀딩비가 높고 변동성이 낮은 조건에서는 진입을 보류하도록 구성해, 정보의 활용 방식을 바꿔 검증했습니다.

검증 결과

필터 적용 전후 비교 백테스트

평가 구간	필터 적용 전 수익률	필터 적용 후 수익률
구간 1	−2.73%	−0.13%
구간 2	−28.24%	−18.04%
구간 3	53.88%	55.72%

선택한 필터의 비교 실험에서 구간 2의 손실률이 10.20%p 줄었으며, 나머지 두 구간에서도 수익률이 개선됐습니다.
※ 연구용 백테스트 결과이며, 실거래 수익이나 향후 성과를 의미하지 않습니다.

배운 점

입력 컬럼의 가치는 정보의 양보다 실제 적용 결과로 판단해야 한다는 점을 배웠습니다. 예상과 다른 결과가 나오면 입력을 계속 늘리기보다, 여러 구간을 비교하고 **해당 정보의 역할을 예측과 위험 제어로 나눠 검증**하는 기준을 세웠습니다.

MAIN PROJECT.3

MOJI

개발 기간 : 2026.03~2026.06

개발 인원 : 4명

담당 역할 : AI 모델 학습 및 평가

주요 담당 : 데이터 검수, 클래스 재정의, 모델 학습·평가, 오분류 분석

모델 : EfficientNet-B0

주요 기술 : Python, PyTorch, torchvision, OpenCV, Pillow

담당 범위 : 데이터 검수, 클래스 재정의, 모델 학습·평가

https://github.com/Okowan00/Advanced_Major_Project.git

https://youtube.com/shorts/TcRAptQX_f0?si=J5Bvo4jwmqoBBtCJ

1. 프로젝트 소개

AI 기반 두피·탈모 자가 모니터링 서비스 MOJI

사용자가 촬영한 정수리·이마 이미지를 AI로 분석하고, 모발 상태와 변화 이력을 확인할 수 있도록 구현한 자가 모니터링 서비스입니다.

프로젝트 설명

MOJI는 사용자가 정수리 또는 이마 라인 이미지를 업로드하면 부위별 AI 모델이 이미지를 분석하고, 예측 단계와 클래스별 확률을 제공하는 서비스입니다. 단순한 이미지 분류 모델 개발에 그치지 않고, 저는 이미지 데이터 검수와 클래스 재정의, 정수리·이마 라인 모델 학습 및 성능 평가를 담당했습니다.

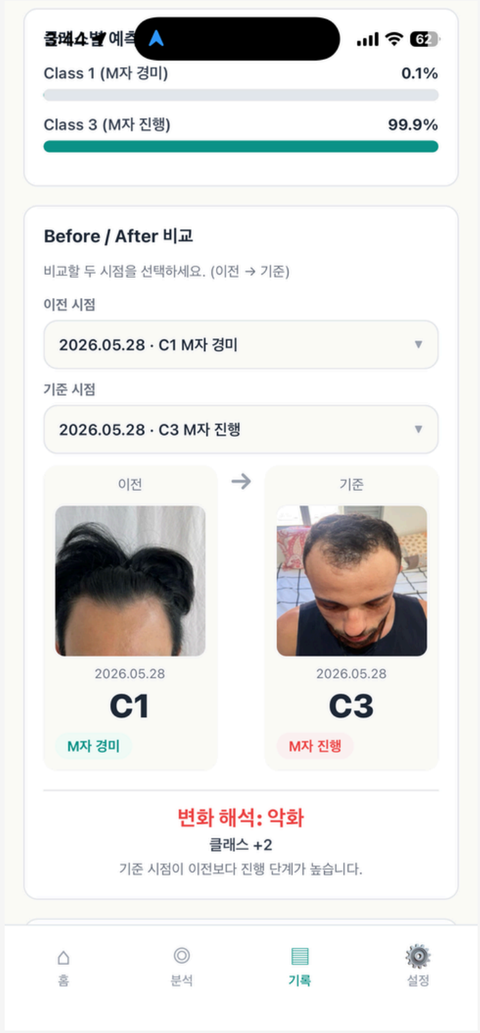
역할 한 줄 요약

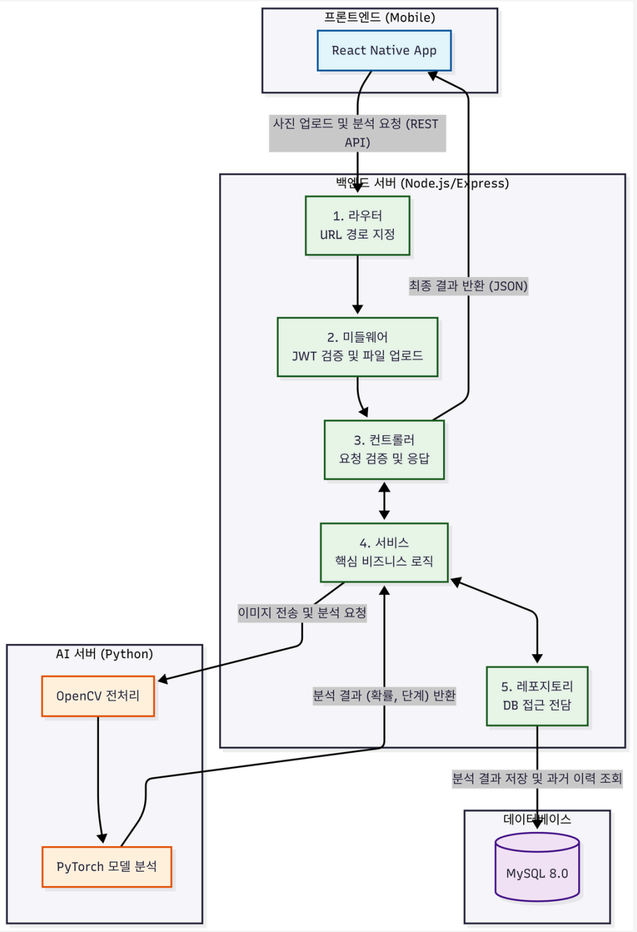
이미지 데이터의 특성을 분석해 분류 문제를 재정의하고, 정수리와 이마 라인에 적합한 부위별 모델을 학습했습니다.

핵심 성과

정수리 모델 Test Accuracy 85.45%

이마 라인 모델 Test Accuracy 86.73%





2.기여한 부분

데이터 검수부터 부위별 모델 학습·평가까지 수행

저는 MOJI에서 AI 모델 개발을 담당했습니다. 주어진 데이터를 그대로 학습하기보다 이미지와 클래스 구조를 직접 검수하고, 정수리와 이마 라인의 특성에 맞게 분류 문제를 재정의한 뒤 부위별 모델을 학습했습니다.

데이터 검수 → 클래스 경계 분석 → 분류 문제 재정의 → 이미지 전처리 → EfficientNet-B0 Fine-tuning → 모델 평가 → 오분류 분석

기여 1. 데이터 검수

- 정수리·이마 이미지의 촬영 부위 확인
- 클래스별 데이터 수와 이미지 특징 비교
- 얼굴·배경이 많이 포함된 이미지 확인
- 흐림, 조명, 각도 차이가 큰 이미지 분석
- 인접 단계 사이의 시각적 경계 검토

기여 2. 클래스 구조 재정의

- 정수리 데이터의 기존 5개 클래스 분석
- 시각적 구분이 모호한 인접 단계 통합
- 정수리 데이터를 3-class로 재구성
- 이마 라인을 정상·의심의 2-class로 구성

기여 3. 모델 학습 및 평가

- 사전학습된 EfficientNet-B0 적용
- 부위별 Classifier 수정
- 이미지 Resize, Tensor 변환, 정규화
- 정수리·이마 라인 모델 개별 학습
- Test Accuracy 측정
- 오분류 이미지 확인 및 원인 분석

3.트러블 슈팅 1/2

모호한 클래스 경계와 제한된 데이터에 맞게 학습 문제를 재정의

정수리 **5-class → 3-class** 재구성
정수리·이마 **부위별 모델 분리**
Test Accuracy **85.45% / 86.73%**

상황

정수리 데이터는 탈모 진행 단계에 따라 5개 클래스로 구분되어 있었습니다. 그러나 이미지를 직접 검수한 결과, level 2와 level 3, level 4와 level 5처럼 **인접 단계 사이의 시각적 차이가 명확하지 않았습니다.**
데이터 규모도 제한적이어서 기존 5개 클래스를 그대로 사용하거나 모델을 처음부터 학습하면 클래스 경계를 안정적으로 학습하기 어렵다고 판단했습니다.

원인

문제 구조
모호한 5-class 경계
제한된 이미지 데이터 → 클래스 간 특징 중첩 → 과적합 및 학습 불안정 가능성

데이터의 시각적 구분 수준보다 클래스가 세분되어 있었고, 정수리와 이마 라인의 이미지 특징도 서로 달랐던 것이 원인이었습니다.

해결

데이터의 시각적 경계와 서비스 목적을 기준으로 **분류 문제를 다시 정의**했습니다.
정수리 모델
level 1
level 2 + level 3
level 4 + level 5
→ 5-class에서 3-class로 재구성

이마 라인 모델
정상 / 의심
→ 2-class 분류
제한된 데이터에서도 안정적으로 학습할 수 있도록 사전학습된 EfficientNet-B0를 적용하고, 정수리 3-class와 이마 라인 2-class에 맞게 Classifier를 각각 변경해 Fine-tuning했습니다.



배운 점

모델 성능은 복잡한 모델을 선택하는 것뿐 아니라, **데이터가 실제로 구분할 수 있는 기준으로 문제를 정의**하는 과정에 크게 영향을 받는다는 점을 배웠습니다.
데이터 특성과 규모를 먼저 분석한 뒤 학습 가능한 클래스 구조와 모델을 선택하는 것이 중요했습니다.

3.트러블 슈팅 2/2

오분류 분석으로 촬영·전처리 개선 기준 도출

오분류 원인 5가지 유형화
촬영·전처리 개선 기준 4개 도출
모델 오류와 입력 품질 문제를 구분

상황

모델 평가 과정에서 일부 이미지가 실제 상태와 다르게 분류됐습니다. 처음에는 모델 성능의 문제로 생각했지만, 오분류 이미지를 직접 확인한 결과 **촬영 부위와 이미지 품질이 일정하지 않은 사례가 반복적으로 발견됐습니다.**

원인

오분류 이미지의 공통 특징

- 촬영 부위 불일치
- 얼굴·배경 영역 과다 포함
- 조명 차이
- 촬영 거리 차이
- 촬영 각도 차이

오분류 사례에서 촬영 부위·배경·조명 등의 차이가 반복적으로 관찰돼, 입력 조건을 함께 점검할 필요가 있다고 판단했습니다.

해결

정확도 수치만 확인하지 않고 **오분류 이미지를 다시 검수**해 입력 조건별로 원인을 분류했습니다. 분석 결과를 바탕으로 모델 학습과 촬영 단계에서 보완해야 할 기준을 정리했습니다.

분석 구조
오분류 이미지 확인
→ 촬영 부위·배경·조명·거리·각도 비교
→ **입력 품질 문제와 모델 판단 문제 구분**
→ **촬영·전처리 개선 기준 도출**

분석 결과

확인한 문제	도출한 개선 기준
촬영 부위 불일치	정수리·이마 촬영 단계 구분
얼굴·배경 과다 포함	두피 중심 ROI Crop
조명·각도·거리 차이	촬영 조건 안내
흐림 이미지	흐림 이미지 재촬영 기준 제안

촬영 조건을 통제하지 않으면 모델 성능만으로 일관된 결과를 보장하기 어렵다는 점을 확인했습니다.

배운 점

이미지 분류에서는 모델 구조뿐 아니라 사용자가 어떤 이미지를 입력하는지도 결과에 큰 영향을 준다는 점을 배웠습니다. 오분류를 단순한 모델 실패로 처리하지 않고 입력 데이터를 함께 분석하면서, **모델 학습과 실제 사용 조건을 함께 고려**해야 한다는 개선 방향을 도출했습니다.

THANK YOU.

AI Backend & Data Developer 서다은 포트폴리오

SEO DAEUN
010-9740-4640
sde4502@naver.com
https://github.com/Seodaeun-hub